

Wellness Tracker

Relatório final — Estruturas de Dados Orientadas a Objetos

Equipe 6: Alexandre Moreno (amrm), Gabriela Lima (gls5), Mario Neto (mmln)

Código: <https://github.com/gabriela-sotero/wellness-tracker>

Página: <https://gabriela-sotero.github.io/wellness-tracker/>

1. O projeto

O Wellness Tracker é um sistema em C++20 para registrar hábitos diários de bem-estar: **água, alimentação, exercício e sono**. Cada hábito gera pontos (XP), os pontos acumulados sobem o nível do usuário e sequências de dias com a meta cumprida (streaks) desbloqueiam medalhas.

É um projeto do **tipo 1** (sistema de informação). O domínio foi modelado com classes próprias, os dados ficam num banco SQLite local e o mesmo núcleo atende duas interfaces: uma de console e uma gráfica em Qt Widgets.

Principais funcionalidades

- Cadastro, login, logout e exclusão de conta; peso opcional para calcular a meta de água (35 ml/kg, ou 2000 ml por padrão).
- Registro diário de água (ml), refeições saudáveis ou não, exercício feito e horas de sono.
- Pontuação por hábito: água e sono até 500 pontos, alimentação e exercício até 300; refeição não saudável desconta 50.
- Progresso de hoje, da semana, do mês e do ano, com a consistência de cada meta.
- Perfil com XP por hábito, nível (o nível n começa em $35 \times (n - 1)^2$ XP), streaks atuais e medalhas de 3 a 365 dias.

2. Arquitetura

O código é dividido em quatro camadas. As interfaces só tratam entrada e saída; as regras ficam no serviço e nos modelos; o SQL fica isolado na classe `Database`. Por isso o console e a GUI reaproveitam o mesmo núcleo (`wellness_core`, uma biblioteca estática no CMake) sem duplicar regra nenhuma.

Arquitetura: interfaces, serviço, modelos e persistência

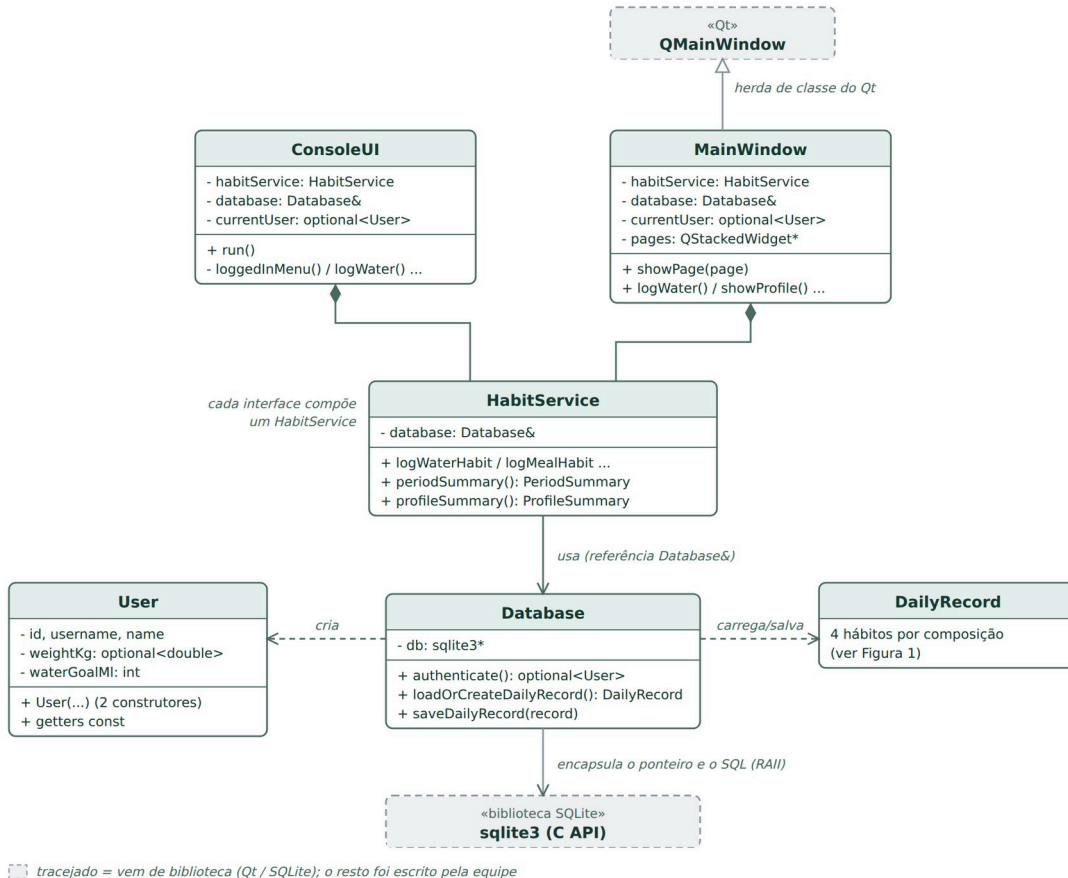


Figura 1 — Classes principais e como se relacionam. Em tracejado, o que vem de biblioteca.

Camada	Classes	Responsabilidade
Interface	ConsoleUI, MainWindow	Menus e telas; validam a entrada e mostram os resultados.
Serviço	HabitService	Registra hábitos, soma períodos, calcula streaks, XP e nível.
Modelo	Habit e subclasses, DailyRecord, User	Estado do usuário e de cada dia; cada hábito calcula a própria pontuação.
Persistência	Database	Abre o SQLite, cria as tabelas e converte linhas em objetos.

O projeto também tem uma suíte de testes unitários (água, alimentação, exercício, DailyRecord e HabitService) e uma ferramenta que gera um perfil de demonstração com um ano de dados.

3. Orientação a objetos no projeto

3.1 Abstração e interface: a classe Habit

Habit é uma classe abstrata que funciona como interface: só declara métodos virtuais puros e não guarda estado. Ela define o contrato que todo hábito precisa cumprir — calcular a

pontuação, informar o progresso (0 a 1) e dizer o próprio nome. Não dá para instanciar um Habit diretamente.

```
class Habit {
public:
    virtual ~Habit() = default;
    virtual int calculateScore() const = 0;
    virtual double progress() const = 0;
    virtual std::string name() const = 0;
};
```

O destrutor virtual garante que, se um hábito for destruído por um ponteiro Habit*, o destrutor da subclasse também é chamado.

3.2 Herança e polimorfismo

Os quatro hábitos herdam publicamente de Habit e sobrescrevem (override) os três métodos. A mesma chamada, calculateScore(), tem regra diferente em cada classe:

Classe	Regra de calculateScore()
WaterHabit	Proporção da meta em ml $\times$ 300, limitada a 100%.
NutritionHabit	Refeições saudáveis / 3 $\times$ 300, menos 50 por refeição não saudável (mínimo 0).
ExerciseHabit	300 se o exercício foi feito, 0 se não.
SleepHabit	Horas dormidas / meta (8 h) $\times$ 300, limitada a 100%.

Hierarquia de hábitos e composição em DailyRecord

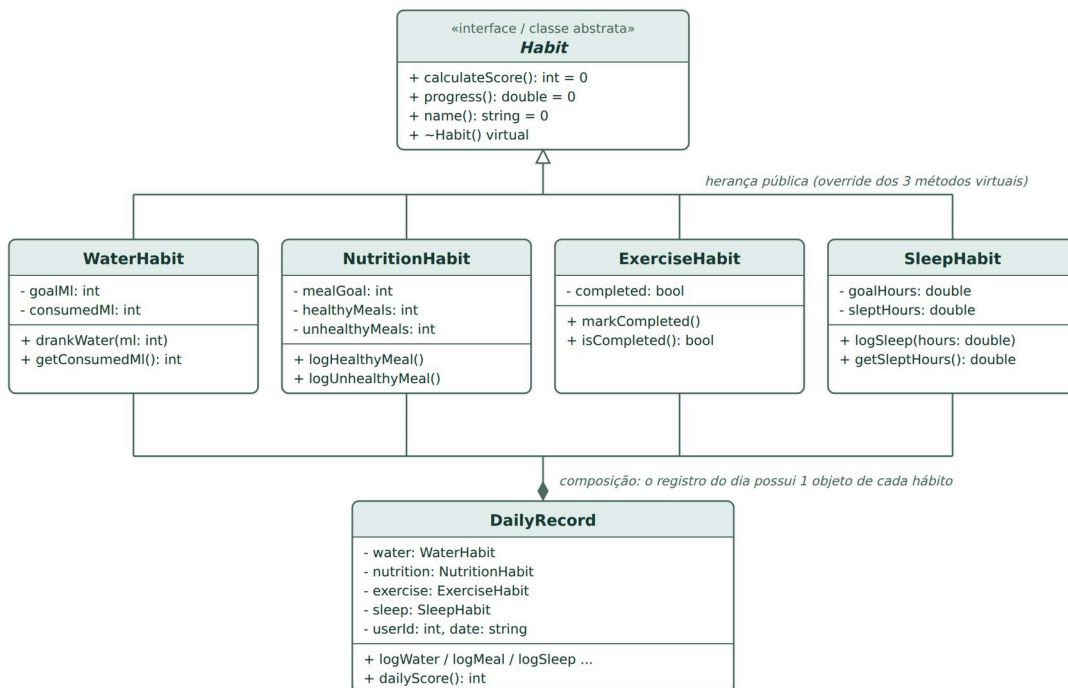


Figura 2 — Hierarquia de Habit e composição em DailyRecord.

O polimorfismo aparece no cálculo da pontuação do dia. `DailyRecord` percorre seus quatro hábitos como ponteiros para `Habit` e chama `calculateScore()` em cada um, sem saber qual hábito é qual. A tabela virtual escolhe, em tempo de execução, a regra da subclasse:

```
int DailyRecord::dailyScore() const {
    const Habit* habits[] = {&water, &nutrition, &exercise, &sleep};
    int total = 0;
    for (const Habit* habit : habits) {
        total += habit->calculateScore();
    }
    return total;
}
```

Para adicionar um hábito novo basta criar outra subclasse de `Habit`, sem mexer na regra das existentes.

3.3 Encapsulamento

Todos os atributos são `private` e só mudam por métodos com significado no domínio. Em `WaterHabit`, por exemplo, não existe `setConsumedMI`: só `drankWater(ml)`, que soma ao total. As classes também validam o próprio estado: `SleepHabit` e `NutritionHabit` lançam `std::invalid_argument` para metas ou horas inválidas. Métodos de leitura são `const`, então o compilador impede que alterem o objeto.

```
class WaterHabit : public Habit {
private:
    int goalMI;
    int consumedMI;
public:
    int calculateScore() const override;
    void drankWater(int ml);    // única forma de alterar o consumo
    int getConsumedMI() const;
};
```

3.4 Composição e associação

- **Composição:** `DailyRecord` possui um objeto de cada hábito. Eles nascem e morrem com o registro do dia, e a pontuação diária é a soma do que cada hábito calcula.
- **Composição:** `ConsoleUI` e `MainWindow` possuem cada uma o seu `HabitService`.
- **Associação:** `HabitService` e as interfaces guardam uma referência `Database&`. Elas usam o banco mas não são donas dele; quem cria e destrói é o `main`.

3.5 Outros conceitos usados

- **Sobrecarga de construtores:** `User` tem um construtor sem peso e outro com peso.
- **RAII:** `Database` abre a conexão no construtor e fecha no destrutor, então a conexão nunca fica aberta por esquecimento.
- **Valores opcionais:** `std::optional<User>` representa "ninguém logado" e `std::optional<double>` o peso não informado, sem números mágicos.
- **Objetos de valor:** structs como `PeriodSummary` e `ProfileSummary` levam os totais prontos do serviço para a tela.

4. O que é nosso e o que vem das bibliotecas

Origem	O que usamos
Escrito pela equipe	Todas as classes do domínio e do serviço (Habit e subclasses, DailyRecord, User, HabitService, Database), as duas interfaces, as regras de pontuação, streaks, níveis e medalhas, os testes e as medalhas em SVG (geradas por script Python).
Qt 6 Widgets	MainWindow herda de QMainWindow e usa Q_OBJECT. As telas são QWidget dentro de um QStackedWidget; botões, campos e barras (QPushButton, QLineEdit, QProgressBar) vêm prontos. Os cliques são ligados com sinais e slots (connect). A memória dos widgets é gerenciada pela árvore pai-filho do Qt.
SQLite 3	API em C (sqlite3_open, sqlite3_prepare_v2...). Por ser C e não ter classes, a encapsulamos na classe Database, que expõe operações com tipos do domínio (User, DailyRecord).
Biblioteca padrão C++	std::string, std::vector, std::optional, <ctime> para datas.

A herança de QMainWindow é o principal ponto de OO que vem do Qt: a nossa classe ganha de graça o comportamento de janela (barra de título, redimensionamento, laço de eventos) e só acrescenta as telas e ações do sistema.

5. O sistema rodando

